Stock Markets Analytics Zoomcamp

by Pythoninvest.com in collaboration with DataTalks.Club

Module 3
Time Series Modeling

Instructor: Ivan Brigida

- Business Intelligence Analyst at Google
- Educational background in Computer Science, Finance, and Data Analytics
- Founder of PythonInvest.com
- Retail investor with a recent interest in fundamental data and options

Agenda

Answer Questions/Comments from Homework 2 ﷺ System Design View (Step 3)

Creating a Unified Dataset from Multiple Sources ("Data Generation & Transformations")

- Defining variable sets
- Create dummy variables for categorical data
- Perform correlation analysis
- Apply temporal split

Time Series modeling

- Rule-based predictions
- Statistical model: ARIMA
- Binary Decision Tree
- Model explainability

Recap

System Design View

Module 3 introduces time series modeling and produces prediction fields

Module 1: Data Sources	Module 2: One Dataframe	Module 3: The Model	Module 4: Trading System	Module 5: Automation

- Output for Module 1: A set of files or tables (if using a database), with each view closely matching the original data source.
- Output for Module 2: A single "flat" file or table containing categorical and numerical features for modeling.
- Output for Module 3: A set of models and prediction fields added to the dataframe.

- https://www.multpl.com/s-p-500-pe-ratio/table/by-month

Vote for questions or submit yours

Join at

https://qna.dtcdev.click/r/sma

Documentation and Q&A from previous years https://datatalks.club/docs/courses/

LINK: https://qna.dtcdev.click/r/sma

Selected Unanswered Questions/Comments

Q1: Time comparison HA1 to HA2
- ❏ HA1: HA2:

(n=148) (n=70)

Q2: Is there a certain amount of points you need to get the certificate?
- ❏ Home assignments are not obligatory, you need to submit a capstone project + review 3 other projects for a Certificate

- ❏ **C1 (HA1)** I totally wasted my first week and did all of that with ai, do not check it unless it is done by the machine opencode + muse 3.1
- ❏ vs. THIS: don't know how to explain it very difficult understand every
- ❏ problem or It takes longer to understand.
- ❏ **C2 (HA1)** result = ticker_obj.get_earnings_dates() - I retrieved 12 rows instead of 25
- ❏ **C3 (HA1)** The homework was interesting and helped me practice financial data analysis with Python and yfinance. I especially enjoyed exploring market corrections and earnings surprise analysis!

- ❏ **C4 (HA2)** Windows/recent pandas+yfinance: " return MultiIndex columns even for single-ticker downloads" ; pandas.read_html(resp.text) =>pandas.read_html(io.StringIO(resp.text))

Add more questions here: https://qna.dtcdev.click/r/sm

```
A few technical friction points worth noting for anyone following this module on
Windows/recent pandas+yfinance versions: yfinance MultiIndex columns: recent
yfinance versions return MultiIndex columns even for single-ticker downloads, so
df["Close"].iloc[0] returns a Series instead of a scalar, breaking float() calls
and arithmetic throughout Module 1 and Module 2. Flattening columns (.squeeze()
or .columns.get_level_values(0)) right after each download avoids this — might
be worth a note in the setup instructions, since it silently produces
TypeError/ValueError in several places. pandas.read_html(resp.text) fails with
an OSError on Windows/newer pandas+lxml — needs
pandas.read_html(io.StringIO(resp.text)) instead. Same fix needed everywhere IPO
data is scraped from HTML tables. ArrowKeyError: A type extension with name
pandas.period already defined when calling .to_parquet() a second time in the
same long-running kernel session (Module 2's final merge/save step) — a known
pandas/pyarrow extension-registration bug. Worth flagging in case others hit it
after re-running that cell during debugging. Lecture notebook's
get_ipos_by_year() pulls from stockanalysis.com, while Homework 2 Q1/Q2
explicitly require iposcoop.com (different site, different data/columns). Took a
moment to realize these weren't the same source — might be worth a one-line
clarification in the homework text for future cohorts. Otherwise, the module
was clear and the data pipeline logic (growth features, TA-Lib indicators,
multi-source merges) came together well once these were resolved.
```

[1/2] Selected Comments from HA2

TO UPDATE

C1: “Basic and advanced knowledge of stock market analytics and its implementation in a dataframe using Pandas is very necessary to complete this second homework assignment. So, more time is needed to participate.”

- ❏ The course builds lecture on lecture: it is crucial that you learn weekly and try to run/adjust the code shared

C2: “Hometasks are very complex. Not for beginners”
“Basic and advanced knowledge of stock market analytics and its implementation in a dataframe using Pandas is very necessary to complete this second homework assignment. So, more time is needed to participate.”

- ❏ Agree, we might write a better prerequisites section and share sample HA tasks before the next iteration starts.
- ❏ I still believe this is the simplest way to achieve a decent quality product and solve end-goal to build a “trading system”

C3: “Really good stuff so far; I specifically value working with the Colab files to see how you structure and execute your code. Thank you!”. “Thanks for the Great Learning Experience. I learned and applied many things through this Module. Although homework questions seemed straightforward, had to struggle sometimes, and think to figure out things.”

- ❏ Great! Keep on trying!

C4: “I'm finding a lot of problems trying to keep up with the content, where should I find information to kinda put myself in line. I'm learning from data camp (financial concepts and other courses) too is it a good idea???”

- ❏ I would use all available means within a limited time frame you have: googling questions, asking experienced friends, asking chatGPT to explain, writing down the gaps to read about later in books

Add more questions here: https://app.sli.do/event/hrRKrRFwRkjvwfdcWb7iY6

[2/2] Selected Comments/Answers from HA2

TO UPDATE

C5: “This module was much better. In the HW we could think about some strategies, this is a big plus. However I did not get what it has to do with IPO and so on. Perhaps it makes sense to build this course as the following: here are some kind of data (ticker from yf, eurostat, etc), what strategy could build using this data? Then we have a technical part, say working with pandas, APIs as well as trading notions, etc. This would give the course a better structure and motivation IMHO”

- ❏ That was exactly the idea how I designed the course with Modules.
- ❏ Modules often intersect: we focus on one topic (data, transformations, modeling, strategies, automation) during each Module + expand previous modules

C6: Some answers on the [Exploratory] question to improve Investing in IPOs:

- ❏ **I would create additional features (binary, numerical):** a) if the company is in a how, growing industry, b) if they have a unique product …
- ❏ “...I can create filters to **evaluate performances of certain verticals and see which industries performed best** over a certain set time. …”
- ❏ “Gathering additional information about each company, such as revenue, debt, liabilities, assets, gross margin and net income… Filter data based on the following statements: **Companies with net income greater than 0** …”
- ❏ “I would look at the **companies profit in comparison to the profit of companies in the same sector** that are already listed”
- ❏ “Perhaps one needs to look at **volume of the stocks** after first day of launch,...”
- ❏ “...Other step would be probably to read news about how is the **market demand and the sentiment** in general for those IPO…”
- ❏ “Yes, blindly investing in IPOs is a bad idea and can lead to bad losses. In general, my approach would be to research about the **company's motive** behind raising capital …”
- ❏ “My strategy is based on b**uying the shares at the time they are released on the market and selling them the same** day at the closing…To know the IPOs in advance, I would analyze the OPIs of **the sectors that have the most successful IPOs on the first day**”
- ❏ ”"I would go with using the **Relative Strength Index (RSI) indicator**, for example I think of set RSI thresholds for buying and selling”
- ❏ “To refine your IPO investment strategy, focus on selecting companies **based on industry trends, historical performance, and sector growth potential**”

Add more questions here: https://app.sli.do/event/hrRKrRFwRkjvwfdcWb7iY6

Final Preparations for the Unified Dataset

Categorical variables, feature name lists, and exploratory analysis

Unified Dataset Organisation: One Row per Date and Ticker

The Primary Key is the pair (Date, Ticker), with all additional features representing everything needed to predict the future growth.

Key feature categories:

- **TO_PREDICT** *(do not include any of this as features)*
  - **is_positive_growth_future_30d** (can be also 1d, 5d, 90d, 225d, etc.)
  - **growth_future_30d** (it can be also 1d, 5d, 90d, 225d, etc.)
- **TO_DROP** *(not to be used as features, but may be useful later)*
  - Year, Date, index_x (artifacts of joins), Quarter
  - **OHLCV**: Exclude original stats, as some are constantly growing
  - **CATEGORICAL:** Exclude text values, but generate dummy variables
- **NUMERICAL** *(all features used, ~200 features)*
  - **GROWTH**: stock growth (lagged/shifted stats or AR=autoregression), growth of other asset types and benchmarks
  - **TECHNICAL INDICATORS and PATTERNS**
  - **MACRO**
  - **CUSTOM_NUMERICAL:** SMA10, SMA20, etc.
  - **[CATEGORICAL] dummies :** month, weekday, ticker?, ticker_type

Key assumptions/considerations for the data:

- Observations are independent (i.i.d), which is a simplification (there is a dependency between tickers and time)
- Some advanced models may require a different data organisation (e.g. Graph Neural Network (GNNs) have edges with ‘relationships”), or one feature can be an array (not an “atomic” type)

Working with Categorical Variables (Generating “Dummies”)

Create one column for each category

Code snippet 1

- **Dummies for:** Month, Weekday, Ticker? (do we need it?), ticker_type are candidates for generating dummies
- Prep: convert Month from datetime (truncated 1st date of month) to string (Month name), and Weekday from Integer to String
- Use `pd.get_dummies()` function to generate dummies. Examples of dummy column names: ***Month_April, Weekday_1, Ticker_ACN***
- **Increases columns count for 184 to 229, and dataframe size from 238Mb to 274 MB.**
- *Do we need that many dummies?(e.g. 33/55 dummies are tickers)*

Correlation analysis

First approximation of the most "correlated" features to be used for predictions

Code snippet 2

… (220 features between)

… (220 features between)

- All features show **weak** correlation values (within the range [-0.08, 0.05]).
- Weak individual features may "stack" together and provide **better predictability/correlation** with a variable from the TO_PREDICT set.
- Use ML **model explainability** to understand what are the **most important metrics** and **identify additional features** to include
- Look for additional features. As noted in Econometrics: *"**Omitted variable bias** occurs when a relevant variable is not included in a regression model, leading to biased and inconsistent estimates"*

Temporal Split by Fixed Intervals (for AMZN and Other tickers)

Time periods: 1. **Training** (fit the model); 2. **Validation** (select hyperparameters);
3. **Test** (evaluate out of sample)

Code snippet 3

- Split the dataset into three segments : Train (usually around 70-80% of data), Validation (10-15%), and Test (10-15%)
- It is important to use a time-based split, not shuffling or other random methods → **otherwise, you may leak future data into your model**

• Try to ensure a similar range of values for all features in Train, Validation,Test sets. E.g. **LICI.NS** has data from 2022-05-17 : it is likely it won't be present in any Train data and the Model won't be properly predicting the 'individual' effect of that particular stock.

AMZN: Checking Distributions for 'Future' Variables by Time Split

There shouldn't be any major structural changes in market dynamics across your data splits

• **The left chart** shows the visually similar distribution for `growth_future_30d.` Important differences in the distributions may not be immediately apparent (solution: see the next slide).

• **The right chart** highlights potential issues for Train/Validation/Test splits regarding `is_positive_growth_30d_future`
The percentage of positive growth days differs significantly between Test and Train, but is more similar between Validation and Train. This could be problematic: while the model may perform well during the Train and Validation periods, it might not capture rapid growth effectively in the Test period.

ALL tickers: Checking Distributions for 'Future' Variables

There shouldn't be any major structural changes in the market dynamics

• **Shape of the distributions:** Are they similar? Check the 25th, 50th, and 75th percentiles (quantiles)
• **Consider removing the top and bottom 1–2% of outliers,** as they may negatively impact estimation for some methods (e.g., RMSE in regression models).

Financial Time Series Modeling

"Hand-crafted" prediction rules
ARIMA models
Binary models: Decision Trees

(Other methods include Logistic/Ordinary Regression, Random Forest,
Neural Network,
Ensemble Learning, etc.)

Predictions using simple rules (not ML)

Code snippet 3

Use "hand rules" and "heuristics" to predict **is_positive_future_growth_30d** using data available today or earlier. This approach may focus on predicting a specific event (e.g. price drop), or use only one/few available features (CCI, RSI, growth_, etc.)

Let's generate a new column with prefix "pred" for each new prediction. This can later be used to select the "best" prediction or even to combine multiple predictions into a new one ("ensemble learning")

Example: a few "hand rules" to predict is_future_growth_30d>0:

- *[1 feature used, 0 params to "estimate"]* **pred0:** CCI>200
- *[1 feature used, 0 params to "estimate"]* **pred1:** growth_30d >1
- *[2 features used, 0 params to "estimate"]* **pred2:** growth_30d >1 & growth_snp_30d >1

Incorrect prediction
(False Positive)

Correct prediction
(True Positive)

Evaluating Prediction Quality (on n=33k test data points) and Comparing Models

Model quality metrics: **Precision (binary)**, Accuracy (binary), RMSE (regression),...
Precision is the ratio of true positive predictions to the total number of positive predictions made by the model
Precision=True Positives / (True Positives+False Positives)

Key considerations when comparing models:

- **Which model is more useful?** (e.g. **520** predictions with 56% precision, or **11k** predictions with 57.8% precision) ? Is this sufficient for profitability?
- Why not **introduce a parameter X** (e.g., CCI>X) and search for a threshold that improves precision? (== Decision tree with a single node)
- Why not use additional features (such as "growth_") to narrow down the prediction space (**and potentially improve the precision**)?
- Will you trade only on positive predictions (where **precision matters most**), or also consider negative predictions (**"accuracy" matters**)?
- How many positive predictions (e.g., investing in growth/"long" trades) are present in the test set? Are these predictions clustered in time or spread out evenly?

ARIMA(p,q,r): AutoRegressive Integrated Moving Average

ARIMA is a **statistical** model that has 3 hyperparams (p,q,r) and **p+q** (~10-20) parameters to estimate

Code snippet 4 - Advanced

Algorithm steps:

- **Find optimal parameters** on the train-validation set (**p=6, q=2, r=2**)
- For each day in the test set:
  - **Retrain** the model using the optimal params and all available information;
  - Predict 1,2,..,**30** periods ahead

(NOTE: confidence intervals widen for each additional period)

  - Compare the current_value and prediction_30periods and make a **prediction** for growth in 30 periods
  - Compare the **prediction** with the actual **is_future_growth_30d**

… 20 more years were used at train+validation

KEY POINTS

- **Idea:** only time-series shape (historical values) is used - no additional features; the model is linear (simple). It is equivalent to generate growth_1d (y_{t-1}),... growth_pd(y_{t-p}) and run a regression $\mathbf{y_t = a_1 * y_{t-1} + a_2 * y_{t-2} + a_3 * y_{t-3} + ... + a_p * y_{t-p}}$ +error (parameters: $a_1 \ldots a_p$ + more)

[2024] Computationally intensive: 500/900 test points for one ticker NVDA took **1h21min** to run. Need to run on 30k test observations to understand the precision, but now it is suspiciously high (**88%**)

- **Good for generic modeling**: for financial planning or forecasting economic indicators (e.g. FED Rates)
- **Assumptions** : ARIMA assumes the data is stationary and provides explainability through confidence
- intervals and parameter importance)

Binary Classification Model: Decision Tree (Random Forest for advanced use)

Idea: Let's select the top **10** (tree_depth hyperparameter) "informative" features (out of ~**200** NUMERICAL + DUMMY features) to predict is_positive_future_growth_30d

Code snippet 5

Level 0. cpi_core_yoy <= 0.023

Level 1. DGS10 <= 1.745 or FEDFUNDS <=4.96

Level 2. DGS1<=0.56 or growth_wti_oil_365d<=1.5 or growth_btc_usd_365d<=1.211 or growth_wti_oil_365d <=0.912 **…. Levels 3..**

10 ⇒ FINAL DECISION

- This model is **non-linear** on params (~can capture complex things), params count is approximately equal to tree depth (**~10-20**). Prone to **overfitting**.
- **Data cleaning:** replace +-inf with something (e.g. ln(volume)=-inf when volume==0 ⇒ replace it with 0)
- **Data cleaning:** do not ignore missing values NaN (replaced with 0 → e.g. "Bitcoin" prices and growth is not defined for 15/25 years)
- **Visual representation:** you can better understand "the decision rules" mechanics when following along Level 0 – Level 10 to predict

Explainability: Feature Importance for a Decision Tree

Use additional features of a model to understand the "decision" process and feature insights

Tree Depth 10 (clf_10 - 2k nodes) vs. 20 (clf_20 - 2m nodes), 1000x more complex!

Precision = 0.598 (current best) Precision = 0.594

- Comparing **feature importance from models** with **simple correlation analysis** helps validate findings, but also reveals the added value of model-based approaches in uncovering deeper patterns and interactions.
- Decision tree with max level 10 highlights macroeconomic indicators and long-term asset growth as the most important features, such as Fedfunds, cpi_core_yoy, growth_gold/wti with a precision of **0.598**
- Decision tree with max level 20 shifts focus toward technical analysis and shorter-term growth metrics, including features like ad, obv, and trix, resulting in a similar precision of **0.594**.

RECAP

What We Learned?

Finalized the unified dataset for modeling
Created "dummies" for categorical variables
Performed a temporal split of the data into training, validation, and test sets
Explored modeling using "hand rules" for binary outcomes
Built and evaluated a statistical ARIMA, introduced the model complexity (±number of parameters)
Developed and assessed decision tree models for binary classification
Out of scope methods: Random Forest (binary), Logistic Regression (binary), Ordinary Regression (numerical), or Neural Network approaches (binary/numerical: non-linear with many parameters)

Module 3

➔ **HA3:** 4 questions + 1 extra
➔ **Deadline** to submit is ~2 weeks from the livestream day, 12 October 2026
➔ **Designed** to be straightforward and relatively easy. You can write feedback in Telegram (under the HA announcement) or when you submit the HA.
➔ **Get extra points** for posting your homework and tagging PythonInvest.com and Datatalks.club on any public platform (e.g. Medium, LinkedIn, X (Twitter), etc.).

Still have questions? Write FAQ (SMA FAQ vs. Data Engineering FAQ), ask in Slack, ask Ivan in Slido